

บทที่ 4

การพัฒนาโปรแกรมระบบจัดการอุปกรณ์ผ่านเครือข่าย

4.1 การพัฒนาส่วนของการติดต่อสื่อสารเพื่อจัดการอุปกรณ์ผ่านซิมเพลเน็ตเวิร์ก เมเนจเมนต์โปรโตคอล (Simple Network Management Protocol Handler)

ในการติดต่อสื่อสารเพื่อจัดการอุปกรณ์ผ่านซิมเพลเน็ตเวิร์กเมเนจเมนต์โปรโตคอลนั้นจะมีด้วยกันอยู่ 2 ส่วนสำคัญ นั่นคือ ส่วนของการติดต่อสื่อสารฝั่งเมเนเจอร์ และ ส่วนของการติดต่อสื่อสารฝั่งเอเจนต์

4.1.1 ส่วนของการติดต่อสื่อสารฝั่งเมเนเจอร์

ในส่วนของการติดต่อสื่อสารฝั่งเมเนเจอร์นั้น มีหน้าที่สำคัญอยู่ 3 หน้าที่ด้วยกันดังนี้

1. ร้องขอข้อมูล และรับข้อมูลจากอุปกรณ์เอเจนต์

เมื่อเมเนเจอร์ต้องการข้อมูลจากเอเจนต์จะทำการร้องขอข้อมูลที่เมเนเจอร์ต้องการไปยังฝั่งเอเจนต์ (GetRequest) จากนั้นเมื่อเอเจนต์ได้รับการร้องขอข้อมูลจากเอเจนต์แล้วจะส่ง ข้อมูลที่เมเนเจอร์ต้องการกลับมา (GetResponse) โดยการร้องขอข้อมูลจะต้องระบุค่าต่างๆให้เรียบร้อยก่อนสิ่งที่ต้องระบุคือ จำนวนออบเจกต์ที่ต้องการร้องขอข้อมูล ออบเจกต์ที่ต้องการร้องขอข้อมูล ชื่อคอมมูนิตี้ออบเจกต์ที่ร้องขอข้อมูล โสตส์เป้าหมายที่ต้องการร้องขอข้อมูล เมื่อทำการระบุเรียบร้อยแล้วจึงทำการส่งการร้องขอข้อมูลไปยังเอเจนต์

โค้ดในส่วนของการร้องขอข้อมูลไปยังเอเจนต์

```
public void sendGet(String deviceIp,String deviceObjId,String
deviceCommunity) {
System.out.println("SNMPManager sendGet : ");
try {
//ระบุจำนวนอบเจ็กต์ที่ต้องการร้องขอข้อมูล
snmp.setObjCount(1);
//ระบุอบเจ็กต์ที่ต้องการร้องขอข้อมูล
snmp.setObjId(1,deviceObjId);           // indexes start at 1
//ระบุชื่อคอมมูนิตีของอบเจ็กต์ที่ร้องขอข้อมูล
snmp.setCommunity(deviceCommunity);
//ระบุโฮสต์เป้าหมายที่ต้องการร้องขอข้อมูล
snmp.setRemoteHost(deviceIp);
//ทำการส่งการร้องขอข้อมูลไปยังโฮสต์เป้าหมาย
snmp.setAction(Snmp.snmpSendGetRequest);
} catch (IPWorksException ipwe) {
JOptionPane.showMessageDialog(null, ipwe.getMessage());
}
}
```

โค้ดในส่วนของการรับข้อมูลที่ตอบกลับจากเอเจนต์

```
void snmp_getResponse(SnmpGetResponseEvent sgre) {
try {
//รับค่าที่ได้จากการตอบกลับของเอเจนต์
String value = new String(snmp.getObjValue(1));
Result = value;
} catch (IPWorksException ipwe) {
JOptionPane.showMessageDialog(null, ipwe.getMessage());
}
//ตรวจสอบว่ามีความผิดพลาดเกิดขึ้นหรือไม่
switch (sgre.errorStatus) {
case 0: result = result; break;
case 1: result = " [ERROR: tooBig (1)]:"; break;
case 2: result = " [ERROR: noSuchName (2)]:"; break;
case 3: result = " [ERROR: badValue (3)]:"; break;
case 4: result = " [ERROR: readOnly (4)]:"; break;
case 5: result = " [ERROR: genError (5)]:"; break;
default: result = " [ERROR: unknown code (" +
sgre.errorStatus + ")]:";
}
}
```

2. ร้อง ขอไปยังเอเจนต์เพื่อทำการปรับเปลี่ยนค่าที่อุปกรณ์เอเจนต์

เมื่อเมนเจอร์ต้องการทำการแก้ไขปรับเปลี่ยนค่า จะทำการร้องขอไปยังเอเจนต์พร้อมทั้งข้อมูลที่ต้องการปรับเปลี่ยนไปยังเอเจนต์ โดยในการร้องขอการปรับเปลี่ยนจะต้องทำการระบุค่าต่างๆ ให้เรียบร้อยก่อน สิ่งที่ต้องระบุคือ จำนวนออบเจกต์ที่ต้องการร้องขอปรับเปลี่ยนค่า ออบเจกต์ที่ต้องการร้องขอปรับเปลี่ยนค่า ชื่อคอมมูนิตี้ออบเจกต์ที่ร้องขอปรับเปลี่ยนค่า โฮสต์เป้าหมายที่ต้องการร้องขอปรับเปลี่ยนค่า ชนิดของออบเจกต์ที่ต้องการปรับเปลี่ยนค่า ค่าที่ต้องการปรับเปลี่ยนโดยต้องทำการเปลี่ยนค่าให้เป็นอาร์เรย์ของไบต์ก่อน เมื่อทำการระบุเรียบร้อยแล้วจึงทำการส่งการร้องขอการปรับเปลี่ยนค่าไปยังเอเจนต์

โค้ดในส่วนของการร้องขอปรับเปลี่ยนค่าไปยังเอเจนต์

```
public void sendSet(String deviceIp,String deviceObjId,String
deviceCommunity,String objType,String objValue) {
    System.out.println("SNMPManager sendSet : ");
    int type = 0;
    try {
        //ทำการตรวจสอบชนิดของอบเจ็กต์
        if (objType.compareTo("otInteger") == 0)
        {
            type = Snmp.otInteger;
        }
        else if (objType.compareTo("otOctetString") == 0)
        {
            type = Snmp.otOctetString;
        }
        else if
        .....
        .....
        .....
        else snmp.setErrorStatus(2);
        //ระบุจำนวนออกเจ็กต์ที่ต้องการปรับเปลี่ยนค่า
        snmp.setObjCount(1);
        //ระบุอบเจ็กต์ที่ต้องการปรับเปลี่ยนค่า
        snmp.setObjId(1,deviceObjId);           // indexes start at 1
        //ระบุชื่อคอมมูนิตีของอบเจ็กต์
        snmp.setCommunity(deviceCommunity);
        //ระบุโฮสต์เป้าหมายที่ต้องการปรับเปลี่ยนค่า
        snmp.setRemoteHost(deviceIp);
        ระบุชนิดของอบเจ็กต์ที่ต้องการปรับเปลี่ยนค่า
        snmp.setObjType(1,type);
        //ระบุค่าที่ต้องการปรับเปลี่ยน โดยต้องทำการเปลี่ยนค่าให้เป็นอาร์เรย์ของไบต์ก่อน
        snmp.setObjValue(1,objValue.getBytes());
        //ทำการส่งการร้องขอการปรับเปลี่ยนค่าไปยังเอเจนต์
        snmp.setAction(Snmp.snmpSendSetRequest);
    } catch (IPWorksException ipwe) {
        JOptionPane.showMessageDialog(null, ipwe.getMessage());
    }
}
```

3. รอรับการแจ้งเตือนข้อผิดพลาดจากเอเจนต์
4. เมื่อเอเจนต์เกิดปัญหาการทำงานผิดพลาดขึ้นหรือมีสถานะบางประการที่จำเป็นต้องแจ้งเข้ามายังเมนเจอร์ ที่ฝั่งเมนเจอร์จะรอรับการแจ้งเตือน (Trap Listener) ที่เอเจนต์จะส่งเข้ามาโดยตัวอย่างโปรแกรมในส่วนนี้จะอยู่ในหัวข้อที่ 4.7

4.1.2 ส่วนของการติดต่อสื่อสารฝั่งเอเจนต์

ในส่วนของการติดต่อสื่อสารฝั่งเอเจนต์นั้น มีหน้าที่สำคัญอยู่ 3 หน้าที่ด้วยกันดังนี้

1. ตอบสนองการร้องขอข้อมูลจากฝั่งเมนเจอร์

เมื่อมีการร้องขอข้อมูลจากฝั่งเมนเจอร์เข้ามายังเอเจนต์ เอเจนต์จะทำการส่งผลลัพธ์กลับมายังเมนเจอร์ โดยหากเอเจนต์นั้นมีออบเจกต์ที่เมนเจอร์ต้องการอยู่จะส่งข้อมูลกลับมาให้ แต่หากไม่มีออบเจกต์นั้นอยู่หรือมีเหตุผลบางประการที่ไม่สามารถส่งข้อมูลกลับมาได้จะทำการส่งกลับมายกยังเมนเจอร์ว่าเพราะเหตุผลใดจึงไม่สามารถส่งข้อมูลกลับมาได้ โดยต้องทำการระบุค่าข้อมูลดังนี้ โฮสต์ระยะไกล พอร์ตระยะไกล ระบุหมายเลขออบเจกต์ ชื่อคอมมูนิตี ชนิดข้อมูล ข้อมูลที่ต้องการตอบสนองกลับไปยังเมนเจอร์ ก่อนที่จะส่งค่าข้อมูลกลับไปยังผู้ร้องขอข้อมูล

โค้ดการตอบสนองข้อมูลไปยังเมนเนเจอร์

```
void snmp1_getRequest(SnmpGetRequestEvent e) {
    try {
        //ระบุโฮสต์ระยะไกลโดยจะทราบจาก e.sourceAddress ซึ่งได้มาจากการร้องขอข้อมูล
        snmp1.setRemoteHost(e.sourceAddress);
        //ระบุพอร์ตระยะไกลโดยจะทราบจาก e.sourceAddress ซึ่งได้มาจากการร้องขอข้อมูล
        snmp1.setRemotePort(e.sourcePort);
        for (int i = 1; i <= snmp1.getObjCount(); i++){
            //รับหมายเลขของอบเจกต์
            snmp1.getObjId(i);
            //ระบุชื่อคอมมูนิตี
            snmp1.setCommunity("public");
            //ระบุชนิดข้อมูล
            snmp1.setObjType(Index, type);
            //ระบุข้อมูลที่ต้องการตอบสนองกลับไปยังเมนเนเจอร์
            snmp1.setObjValue(Index, value.getBytes());
        }
        //ส่งค่าข้อมูลกลับไปยังผู้ร้องขอข้อมูล
        snmp1.sendGetResponse();
    } catch (IPWorksException ipwe) {
        JOptionPane.showMessageDialog(this, ipwe.getMessage());
    }
    try {
        snmp1.reset();
    } catch (IPWorksException ipwe) {
    }
}
```

2. ปรับเปลี่ยนข้อมูลตามการร้องขอปรับเปลี่ยนข้อมูลจากฝั่งเมนเนเจอร์

เมื่อมีการร้องขอปรับเปลี่ยนข้อมูลจากฝั่งเมนเนเจอร์เข้ามายังเอเจนต์ จะทำการปรับเปลี่ยนข้อมูลให้ตามที่เมนเนเจอร์ต้องการโดยทำการรับหมายเลขของอบเจกต์ และ ค่าข้อมูลที่เมนเนเจอร์ต้องการปรับเปลี่ยน

```

โค้ดแสดงการรับค่าข้อมูลเพื่อทำการปรับเปลี่ยนค่าจากเมนเจอร์
void snmp1_setRequest(SnmpSetRequestEvent e) {
    try {
        for (int i = 1; i <= snmp1.getObjCount(); i++) {
            //รับหมายเลขอบเจกต์
            snmp1.getObjId(i);
            //รับค่าที่ทำการปรับเปลี่ยนมาจากเมนเจอร์
            String value = new String(snmp1.getObjValue(i));
            .....
            .....
        }
    } catch (IPWorksException ipwe) {
        JOptionPane.showMessageDialog(this,
ipwe.getMessage());
    }

    try {
        snmp1.reset();
    } catch (IPWorksException ipwe) {
    }
}

```

3. แจ้งเตือนไปยังเมนเจอร์เมื่อเกิดสภาวะผิดปกติหรือมีเหตุการณ์บางอย่างที่ต้องแจ้งไปยังเมนเจอร์
4. เมื่อมีการทำงานผิดปกติเกิดขึ้นหรือมีเหตุการณ์บางประการที่จำเป็นต้องแจ้งไปยังเมนเจอร์ เอเจนต์จะทำการส่งข้อมูลไปยังเมนเจอร์ (Trap) ซึ่งเมนเจอร์จะรอรับการแจ้งเตือนตลอดเวลา (Trap Listener)

```

void buttonSendTrap_actionPerformed(ActionEvent e) {
    boolean initialState = snmp1.isActive();
    try {
        //ระบุอบเจ็กต์ที่ทำการแจ้งเตือน
        snmp1.setTrapEnterprise(txtOIDVal1.getText());
        //ระบุชนิดการแจ้งเตือนแบบทั่วไป
        snmp1.setTrapGenericType(Snmp.tgtEnterpriseSpecific);
        //ระบุชนิดการแจ้งเตือนแบบเฉพาะ
        snmp1.setTrapSpecificType(777);
        //ระบุเวลาในการส่งการแจ้งเตือน
        snmp1.setTrapTimeStamp(Integer.parseInt(txtOIDVal2.getText()));
        //ระบุโฮสต์ระยะไกล
        snmp1.setRemoteHost(remoteHost.getText());
        //ทำการแจ้งเตือนไปยังโฮสต์ระยะไกล
        snmp1.sendTrap();
    } catch (IPWorksException ipwe) {
        JOptionPane.showMessageDialog(this, ipwe.getMessage());
    }

    if (!initialState) try {
        snmp1.setActive(false);
    } catch (IPWorksException ipwe) {
    }
}

```

4.2 การพัฒนากลไกในการจัดการเมเนจออบเจ็กต์ของอุปกรณ์ (Message Handler)

เนื่องด้วยการออกแบบระบบจัดการอุปกรณ์ผ่านเครือข่ายนั้น ไม่ต้องการให้ระบบยึดติดกับการสื่อสารผ่านซีมเพลเน็ตเวิร์กเมเนจเมเนตโปนโตคอลเพียงโปรโตคอลเดียว ดังนั้นจึงได้ทำการออกแบบ กลไกในการที่ส่วนของโปรแกรมที่มีหน้าที่ประมวลผล แสดงผล ต้องการติดต่อกับอุปกรณ์ โดยใช้หลักการของการสร้างข้อความ (message) เพื่อใช้สื่อสารเพื่อให้ส่วนประกอบต่าง ๆ ในโปรแกรมมีรูปแบบในการติดต่อกับอุปกรณ์เป็นไปในรูปแบบเดียวกัน

```

public void constructMessage(String desadd,String desobj,String
oper,String dfo,String dt,String value)
{ destinationdevice=desadd;
  destinationobject=desobj;
  operation=oper;
  dataforoperation=dfo;
  datatype=dt;
  datavalue = value;
  translateMessage();}

```



```

public void translateMessage()
{

    ResultSet rs = db.executeQuery("SELECT IP,PROTOCOL FROM
    DEVICE WHERE DEVICEID='"+destinationdevice+"'");
    try{
        while(rs.next())
        {

            hostname = rs.getString("IP");
            protocolname = rs.getString("PROTOCOL");
        }
    } catch(SQLException sql)
    {
        System.out.println("Error connecting Database");
    }
    System.out.println(hostname);

    if (destinationobject.compareTo("") == 0)
    {
        objectid = "1.3.6.1.2.1.1.1.0";
        community = "public";
    }
    else
    {
        ResultSet rs1 = db.executeQuery("SELECT
        SNMPOBJECTID,COMMUNITY FROM SNMPOBJECT WHERE
        OBJECTID='"+destinationobject+"' AND DEVICEID = '"
        +destinationdevice+"'");
        try{
            while(rs1.next())
            {
                objectid = rs1.getString("SNMPOBJECTID");
                community = rs1.getString("COMMUNITY");
            }
        } catch(SQLException sql)
        {
            System.out.println("Error connecting Database");
        }
    }
}

```

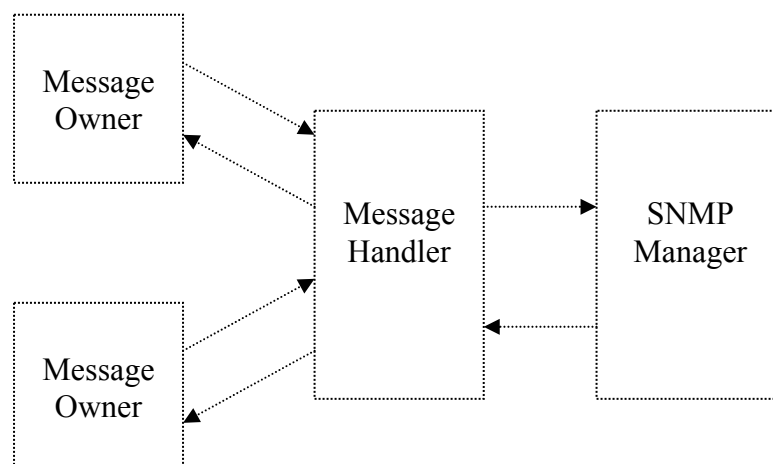
โค้ดข้างต้นแสดงการทำงานเพื่อแปลข้อมูลที่ได้มาจากผู้เรียกโดยการดึงข้อมูลที่จำเป็น
จากฐานข้อมูลเพื่อทำการประมวลผล เพื่อทำงานต่อไป

```

public void sendMessage()
{
    if(operation.compareTo("GET")==0)
    {
        if(protocolname.compareTo("SNMP")==0)
        {
            snmp.sendGet(hostname,objectid,"public");
        }
    }
    else if (operation.compareTo("SET") == 0)
    {
        if (protocolname.compareTo("SNMP") == 0)
        {
            snmp.sendSet(hostname,objectid,community,datatype,datavalue);
        }
    }
}

public String recieveMessage()
{
    if(protocolname.compareTo("SNMP")==0)
    {
        snmpresult = snmp.getResult();
        System.out.println(snmresult);
    }
    return snmresult;
}

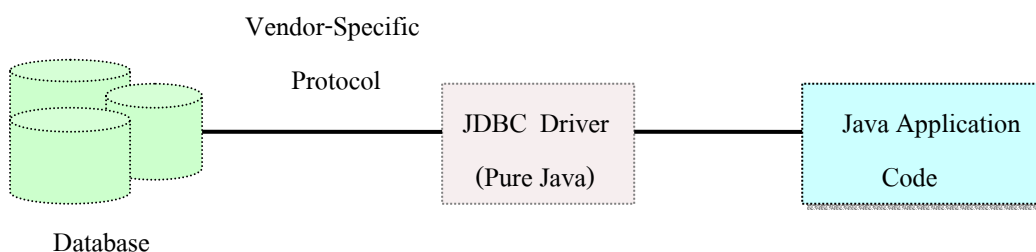
```



รูป 4-1 การทำงานของเมสเสจแฮนเดเลอร์

4.3 การพัฒนาส่วนของการติดต่อกับระบบจัดการฐานข้อมูล (Database Management System Connection)

ในการพัฒนาส่วนการติดต่อกับระบบจัดการฐานข้อมูลนั้น ได้ทำการใช้การติดต่อแบบชนิดที่ 4 คือทำการติดต่อผ่าน ไดรฟ์เวอร์ JDBC ที่เป็นการเชื่อมต่อในลักษณะที่เป็นโปรโตคอลของแต่ละบริษัท ในโครงการนี้ได้นำ Microsoft SQL Server มาใช้เป็นระบบจัดการฐานข้อมูลจึงใช้ `com.microsoft.jdbc.sqlserver.SQLServerDriver` ซึ่งเป็นไดรฟ์เวอร์ของไมโครซอฟท์โดยเฉพาะ เพื่อไม่ต้องทำการติดต่อผ่าน ODBC ดังนั้นเมื่อนำโปรแกรมนี้ไปติดตั้งลงเครื่องที่ต้องการจะไม่จำเป็นต้องทำการปรับแต่งค่าที่ ODBC อีก



รูป 4-2 ลักษณะการเชื่อมต่อระหว่างโปรแกรมกับฐานข้อมูล

ในการติดต่อไปยังระบบฐานข้อมูลนั้นจะทำการเชื่อมต่อเพียงครั้งแรกของการใช้งานโปรแกรม หลังจากนั้นส่วนที่ต้องการทำงานใดๆกับฐานข้อมูลจะทำการเรียกการเชื่อมต่อที่ได้ทำไว้แล้วมาใช้ในการรับส่งข้อมูล โดยในการเชื่อมต่อครั้งแรกนั้นจะทำการส่งชื่อและรหัสผ่านไปยังเครื่องเซิร์ฟเวอร์ฐานข้อมูล

โค้ดแสดงการเชื่อมต่อไปยังเครื่องเซิร์ฟเวอร์ฐานข้อมูล

```
public Database(String ip,String username,String password) {
    try
    {
        //ระบุไดรฟ์เวอร์
        Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver");
        //สร้างการเชื่อมต่อโดยระบุตำแหน่งเครื่อง ชื่อ และ รหัสผ่าน
        dbCon = DriverManager.getConnection
        ("jdbc:microsoft:sqlserver://" + ip + ":1433",username,password);

    }
    .....
    .....
}
```

โค้ดแสดงการอ่านข้อมูลจากฐานข้อมูล

```
public ResultSet executeSQLquery(String sql)
{
    try{
        //สร้างสแตตเมนต์
        Statement s = dbCon.createStatement();
        //ทำการอ่านข้อมูลจากฐานข้อมูล
        ResultSet rs = s.executeQuery(sql);
    }
    .....
    .....
    return rs;
}
```

โค้ดแสดงการเขียนข้อมูลไปยังฐานข้อมูล

```
public void executeSQLupdate(String sql)
{
    try{
        Statement s = dbCon.createStatement();
        s.executeUpdate(sql);
    }
    catch (SQLException sqle)
    {
        System.out.println("Error connecting DB");
        System.out.println(sqle.toString());
    }
}
```

4.4 การพัฒนาส่วนของการพิสูจน์ตนเข้าใช้งานระบบ , จำแนกสิทธิ และ การบันทึกการเข้าใช้งานระบบ(Authentication, Authorization, Audits)

```
ResultSet rs = db.executeSQLquery("SELECT USERPASSWORD
FROM USERTABLE WHERE USERID='"+check+"' AND
USERTYPE='user'");
while (rs.next())
{ss = rs.getString("USERPASSWORD");}
.....
if (trans.length!=check2.length)
{System.out.println("Password not correct.");
dialog.annouce("Password incorrect, Please re-enter password");
dialog.show();}
else
{ for(int i=0;i<trans.length;i++)
{ if (trans[i]!=check2[i])
ck=false; }
if (ck==false){
System.out.println("Password not correct.");
dialog.annouce("Password incorrect, Please re-enter password");
dialog.show();}
else if (ck==true)
{
System.out.println("Password correct");
db.executeSQLupdate("INSERT INTO EVENT
(EVENTNAME,EVENTTYPEID,EVENTDESCRIPTION,EVENTDATE,EVENTTIME)VALUES ('Userlogin','01','Userlogin to project','+(new
Date(System.currentTimeMillis().toString())+'',(new
Time(System.currentTimeMillis().toString())+'"))");
```

4.5 การพัฒนาส่วนการตั้งค่าการใช้งาน (Device management Configuration)

ในการพัฒนาส่วนการตั้งค่าสำหรับการจัดการ จะแบ่งส่วนหลักๆ ได้ 2 ส่วน ในส่วนที่หนึ่ง คือ ส่วนการตั้งค่าในตอนเริ่มต้นครั้งแรก และส่วนที่สองเป็นส่วนการตั้งค่าในลักษณะการปรับเปลี่ยนข้อมูลการตั้งค่าจากในส่วนที่หนึ่ง

4.5.1 ส่วนการตั้งค่าการใช้งานเริ่มต้น (Configuration)

ในส่วนของการตั้งค่าการใช้งานเริ่มต้นนั้นจะมีส่วนหลัก 3 ส่วนคือ การตั้งค่าผู้ใช้ การตั้งค่าอุปกรณ์ การตั้งค่าโปรเจก

4.5.1.1 การตั้งค่าผู้ใช้ (User Configuration)

การทำงานในส่วนของการตั้งค่าผู้ใช้จะเป็นการใส่ข้อมูลของผู้ใช้เก็บลงฐานข้อมูล ผู้ใช้แต่ละคนจะมีรหัสประจำตัว ชื่อ รหัสผ่าน แพนก และประเภท เมื่อต้องการเก็บข้อมูลของผู้ใช้เข้าสู่ระบบ จะเรียกการทำงานในส่วนนี้ โดยข้อมูลทั้งหมดเหล่านี้จะทำการเก็บลงฐานข้อมูล ซึ่งตารางที่เกี่ยวข้อง ได้แก่ USERTABLE

4.5.1.2 การตั้งค่าอุปกรณ์ (Device Configuration)

การทำงานในส่วนของการตั้งค่าอุปกรณ์จะมีการใส่ข้อมูลของอุปกรณ์นั้น โดยจะมีข้อมูลอุปกรณ์ รหัสอุปกรณ์ โปรโตคอลที่อุปกรณ์ใช้ในการสื่อสาร ออบเจกต์ที่อุปกรณ์มี ไอพีแอดเดรส ไฟล์มิม การจับคู่ระหว่างออบเจกต์ของอุปกรณ์กับออบเจกต์ในเอสเอ็นเอ็มพี โดยข้อมูลทั้งหมดเหล่านี้จะทำการเก็บลงฐานข้อมูล ซึ่งตารางที่เกี่ยวข้อง ได้แก่ DEVICE, OBJECT, SNMPOBJECT

4.5.1.3 การตั้งค่าโปรเจก (Project Configuration)

การทำงานในส่วนของการตั้งค่าโปรเจก จะมีการใส่ข้อมูลของโปรเจกโดยจะมี รหัสของโปรเจก ชื่อโปรเจก รายละเอียดโปรเจก และวันที่ที่ตั้งค่าโปรเจก เลือกอุปกรณ์ที่ใช้ในโปรเจกนี้ เลือกออบเจกต์ที่ต้องการ ทำการตั้งค่ากลุ่มในการมอนิเตอร์ ว่ามีอุปกรณ์ใด ออบเจกต์ใดบ้าง และกำหนดประเภทในการมอนิเตอร์ ทำการตั้งค่าการตรวจสอบความผิดพลาดว่ามีอุปกรณ์ใด ออบเจกต์ใดบ้าง และกำหนดประเภทในการตรวจสอบความผิดพลาด ทำการตั้งค่าการปรับเปลี่ยนข้อมูลว่ามีอุปกรณ์ใด ออบเจกต์ใดบ้าง เลือกผู้ใช้ที่สามารถทำงานร่วมกับโปรเจกนี้ โดยข้อมูลทั้งหมดเหล่านี้จะทำการเก็บลงฐานข้อมูล ซึ่งตารางที่เกี่ยวข้อง ได้แก่ PROJECT , PROJECTDEVICE, PROJECTUSER, MONITOR, PROJECTGROUP, SUPPORTGROUP, MONITORGROUP, FAULT, TUNING

4.5.2 ส่วนการปรับเปลี่ยนการตั้งค่า (Edit Configuration)

ในส่วนของการปรับเปลี่ยนการตั้งค่านั้นมีส่วนประกอบด้วยกันอยู่ 9 ส่วนดังนี้

4.5.2.1 การปรับเปลี่ยนการตั้งค่าโปรเจกต์ (Edit Project Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่าโปรเจกต์สามารถทำการแก้ไขข้อมูลโปรเจกต์เดิมและลบข้อมูลโปรเจกต์เดิม ในการแก้ไขจะเลือกตามรหัสโปรเจกต์ ค่าที่สามารถแก้ไขได้ คือ ชื่อโปรเจกต์ รายละเอียดโปรเจกต์ วันที่ที่ตั้งโปรเจกต์ ตารางที่เกี่ยวข้องคือตาราง PROJECT

4.5.2.2 การปรับเปลี่ยนการตั้งค่าผู้ใช้ (Edit User Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่าผู้ใช้สามารถทำการแก้ไขข้อมูลผู้ใช้เดิมและลบข้อมูลผู้ใช้เดิม ในการแก้ไขจะเลือกตามรหัสผู้ใช้ ค่าที่สามารถแก้ไขได้ คือ ชื่อผู้ใช้ แผนกรหัสผ่าน และประเภท ตารางที่เกี่ยวข้องคือตาราง USERTABLE

4.5.2.3 การปรับเปลี่ยนการตั้งค่าอุปกรณ์ (Edit Device Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่าอุปกรณ์สามารถทำการแก้ไขข้อมูลอุปกรณ์เดิมและลบข้อมูลอุปกรณ์เดิม ในการแก้ไขจะเลือกตามรหัสอุปกรณ์ ค่าที่สามารถแก้ไขได้ คือ ชื่ออุปกรณ์ ไอพีแอดเดรส โปรโตคอล และไฟลด์มีบ ตารางที่เกี่ยวข้องคือตาราง DEVICE

4.5.2.4 การปรับเปลี่ยนการตั้งค่าออบเจกต์ (Edit Object Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่าออบเจกต์สามารถทำการเพิ่มออบเจกต์ แก้ไขข้อมูลออบเจกต์ และลบออบเจกต์ โดยทำการเลือกตามรหัสอุปกรณ์ ในส่วนของการเพิ่มสามารถกรอกข้อมูลที่ต้องการเพิ่มลงไปได้เลย แต่หากทำการแก้ไขและลบ จะต้องทำการเลือกออบเจกต์ก่อน ตารางที่เกี่ยวข้องคือตาราง OBJECT , SNMPOBJECT

4.5.2.5 การปรับเปลี่ยนการตั้งค่าผู้ใช้ในโปรเจกต์ (Edit User In Project Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่าผู้ใช้ในโปรเจกต์ สามารถทำการเพิ่มผู้ใช้ และลบผู้ใช้ที่ทำงานในโปรเจกต์ โดยทำการเลือกจากรหัสโปรเจกต์ และทำการเลือกว่าต้องการให้มีผู้ใช้คนใดทำงานกับโปรเจกต์นี้บ้าง ตารางที่เกี่ยวข้องคือตาราง PROJECTUSER

4.5.2.6 การปรับเปลี่ยนการตั้งค่าอุปกรณ์ในโปรเจกต์ (Edit Device In Project Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่าอุปกรณ์ในโปรเจกต์ สามารถทำการเพิ่มอุปกรณ์ และลบอุปกรณ์ที่ทำงานในโปรเจกต์ โดยทำการเลือกจากรหัสโปรเจกต์ และทำการเลือกว่าต้องการให้มีอุปกรณ์ใดทำงานกับโปรเจกต์บ้าง ตารางที่เกี่ยวข้องคือตาราง PROJECTDEVICE

4.5.2.7 การปรับเปลี่ยนการตั้งค่ามอนิเตอร์ (Edit Monitor Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่ามอนิเตอร์ สามารถทำการเพิ่ม แก้ไข และลบกลุ่มของการมอนิเตอร์ โดยทำการเลือกจากรหัสโปรเจกต์และรหัสของกลุ่มการมอนิเตอร์ จากนั้นทำการเพิ่มลดออบเจกต์ที่ต้องการมอนิเตอร์ตามต้องการ ตารางที่เกี่ยวข้องคือตาราง MONITOR , PROJECTGROUP , MONITORGROUP , SUPPORTGROUP

4.5.2.8 การปรับเปลี่ยนการตั้งค่าการปรับแต่งข้อมูล (Edit Tuning Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่าการปรับแต่งข้อมูล สามารถทำการเพิ่ม และลบออบเจกต์ที่ต้องการปรับแต่งข้อมูลได้ โดยทำการเลือกจากรหัสโปรเจกต์ และทำการเพิ่มลดออบเจกต์ที่ต้องการปรับแต่งข้อมูลตามต้องการ ตารางที่เกี่ยวข้องคือตาราง TUNING

4.5.2.9 การปรับเปลี่ยนการตั้งค่าการตรวจสอบความผิดพลาด (Edit Fault Configuration)

ในส่วนการปรับเปลี่ยนการตั้งค่าการตรวจสอบความผิดพลาด สามารถทำการเพิ่ม แก้ไขและลบออบเจกต์ที่ต้องการตรวจสอบความผิดพลาดได้ โดยทำการเลือกจากรหัสโปรเจกต์และรหัสการตรวจสอบความผิดพลาด จากนั้นทำการเพิ่มลดออบเจกต์ที่ต้องการตรวจสอบความผิดพลาดและเลือกประเภทที่ต้องการตรวจสอบได้ตามต้องการ ตารางที่เกี่ยวข้องคือตาราง FAULT

4.6 การพัฒนาส่วนการตรวจตราและปรับปรุงการทำงานของอุปกรณ์ (Monitoring and tuning)

4.6.1 ส่วนการตรวจตราการทำงานของอุปกรณ์ (Monitoring)

ในการพัฒนาส่วนของการตรวจตราการทำงานของอุปกรณ์ จะเริ่มจากการตรวจสอบอุปกรณ์ว่ามีการเชื่อมต่ออยู่บนระบบเครือข่ายหรือไม่ โดยการเข้าไปเพื่อตรวจตราอุปกรณ์สามารถทำได้ 2 ทางคือเข้าไปที่อุปกรณ์นั้นๆ แล้วเลือกอบเจกต์ หรือ เลือกตามกลุ่มที่ได้จัดไว้ในการตั้งค่าโปรเจก

4.6.1.1 การแสดงผลในรูปแบบกราฟ (GRAPH)

การแสดงผลในรูปแบบกราฟนั้นทำงานในลักษณะที่สามารถแสดงผลแบบเปลี่ยนค่าได้ (Dynamic Graph) โดยการทำงานจะเรียกใช้แมสเชสเซนเดอร์ให้ทำการดึงข้อมูลจากอุปกรณ์ หลังจากนั้นทำการกำหนดตำแหน่งจุดลงไปในกราฟ

```

protected void addNewPoints(double Hold, int expectedLastIndex,
                             int thisNumberOfPoints)
{
    for (int i = currentLastIndex + 1;
         i < thisNumberOfPoints && i <= expectedLastIndex; i++) {
        currentLastPoint++;
        xvalues[0][i] = currentLastPoint;
        for (int j=0;j<num;j++)
        {

m.constructMessage(deviceobjectid[j][0],deviceobjectid[j][1],"GET","","","");
        m.sendMessage();
        try
        {
            Thread.currentThread().sleep(100);
        }
        catch (Exception e)
        {
            System.out.println(e);
        }
        String value = m.recieveMessage();
        if (value.compareTo("*") == 0)
        {
            //หากไม่มีการตอบสนองมาจากอุปกรณ์ทำการกำหนดค่าเป็นศูนย์
            yvalues[j][i] = 0;
        }
        else
        {
            //หากมีการตอบสนองจากอุปกรณ์ทำการแปลงเป็นเลขทศนิยมแล้วกำหนดค่าไปยังกราฟ
            yvalues[j][i] = Double.parseDouble(value);
        }

        }
        currentLastIndex++;
    }
}

```

4.6.1.2 การแสดงผลในรูปแบบตาราง (TABLE)

การแสดงผลในรูปแบบตารางนั้นทำงานในลักษณะที่สามารถแสดงผลแบบเปลี่ยนค่าได้ (Dynamic Table) โดยการทำงานจะเรียกใช้เมสเสจสเอนเดอร์ให้ทำการดึงข้อมูลจากอุปกรณ์ หลังจากนั้นทำการใส่ค่าไปยังตารางตามหลักที่ต้องการ ลักษณะในการทำงานจะทำการเรียกเทรด(Thread) ขึ้นมาทำการรับข้อมูลจากอุปกรณ์แล้วจึงทำการส่งค่าที่ได้รับจากอุปกรณ์ไปยังตาราง

โค้ดแสดงการรับข้อมูลจากอุปกรณ์ซึ่งเป็นเทรด (Thread) แล้วจึงส่งค่าไปยังหน้าต่างหรือหน้าต่างหนังสือ

```
public void run()
{
    while (true)
    {
        System.out.println(numofdevice);
        for (int j=0;j<numofdevice;j++)
        {
            m.constructMessage(deviceobjectid[j][0],deviceobjectid[j][1],"GET","", "", "");
            m.sendMessage();
            try
            {
                Thread.currentThread().sleep(500);
            }
            .....
            value[0] = deviceobjectid[j][0];
            value[1] = deviceobjectid[j][1];
            //รับข้อมูลจากอุปกรณ์
            value[2] = m.recieveMessage();
            if (value[2].compareTo("*") == 0)
            {
                //ไม่มีการตอบสนองจากอุปกรณ์
                value[2] = "not response";
            }
            if (type.compareTo("text") == 0)
            {
                //หากผู้เรียกเทรดเป็นแบบตัวหนังสือจะทำการกำหนดค่าไปยังหน้าต่างตัวหนังสือ
                frametext.setvalue(value);
            }
            else if (type.compareTo("table") == 0)
            {
                //หากผู้เรียกเทรดเป็นแบบตารางจะทำการกำหนดค่าไปยังหน้าต่างตาราง
                frametable.setvalue(value);
            }
        }
        if (type.compareTo("text") == 0)
        {
            //หากผู้เรียกเทรดเป็นแบบตัวหนังสือจะทำการสั่งให้แสดงผลไปยังหน้าต่างตัวหนังสือ
            frametext.write();
        }
        else if (type.compareTo("table") == 0)
        {
            //หากผู้เรียกเทรดเป็นแบบตารางจะทำการสั่งให้แสดงผลไปยังหน้าต่างตาราง
            frametable.write();
        }
    }
}
```

4.6.1.3 การแสดงผลในรูปแบบตัวหนังสือ (TEXT)

การแสดงผลในรูปแบบตัวหนังสือนั้นทำงานในลักษณะที่สามารถแสดงผลแบบเปลี่ยนค่าได้ (Dynamic Text) โดยการทำงานจะเรียกใช้เมสเสจเซนเดอร์ให้ทำการดึงข้อมูลจากอุปกรณ์ หลังจากนั้นทำการใส่ค่าไปยังพื้นที่ใส่ข้อความตามต้องการ ต้องการ ลักษณะในการทำงานจะทำการเรียกเทรด (Thread) ขึ้นมาทำการรับข้อมูลจากอุปกรณ์แล้วจึงทำการส่งค่าที่ได้รับจากอุปกรณ์ไปยังหน้าตัวหนังสือ (ทำการเรียกเทรดตัวเดียวกันกับที่ตารางเรียก)

4.6.2 ส่วนการปรับปรุงการทำงานของอุปกรณ์ (Tuning)

การทำงานในส่วนการปรับปรุงการทำงานของอุปกรณ์จะสามารถทำการกำหนดค่าไปยังอุปกรณ์ได้โดยทำการตั้งค่าว่ามีอบเจกต์ไหนที่สามารถทำการกำหนดค่าไปยังอุปกรณ์ได้บ้างไว้ก่อน เมื่อต้องการปรับปรุงการทำงานของอุปกรณ์ จะทำการสร้างเมสเสจสว่าให้ทำการปรับแต่งค่าไปยังอุปกรณ์ใด อบเจกต์ใด และค่าคืออะไร แล้วจึงทำการส่ง

```
โค้ดแสดงการปรับปรุงการทำงานของอุปกรณ์
void Set_actionPerformed(ActionEvent e) {
    if (Value.getText().compareTo("") != 0)
    {
        //สร้างเมสเสจสว่าให้ทำการปรับแต่งค่าไปยังอุปกรณ์ใด อบเจกต์ใด และค่าคืออะไร
        m.constructMessage(parent,leafpath,"SET" ,"",datatype,Value.getText());
        //ส่งค่าไปยังอุปกรณ์
        m.sendMessage();
    }
    else
    {
        Objectdetail.append("Please insert value into the field");
    }
}
```

4.7 การพัฒนาส่วนการจัดการความผิดปกติ(Fault Management)

จากการที่แสดงถึงการทำงานของส่วนจัดการความผิดปกติ ไว้ในบทที่กล่าวถึงการออกแบบ ในการพัฒนาขั้นแรกเราจำเป็นต้องทำการดึงการตั้งค่าที่เกี่ยวข้องกับการจัดการความผิดพลาด ที่ได้กระทำไว้ในตอนแรกก่อนใช้งาน

```
public class FaultListener extends Thread {
```

ทำการประกาศคลาสที่จะทำหน้าที่ในการโพลลิงไปยังอุปกรณ์ต่าง ๆ เป็นเทรด (Thread) เพื่อที่คอยทำหน้าที่ในการตรวจสอบความผิดพลาด

```

public FaultListener(String project,String re,boolean at,Date sd,Time st,Date
ed,Time et) {
.....
ResultSet rs3 = db.executeQuery("SELECT * FROM FAULT WHERE
PROJECTID='"+listenproject+"'");
.....
ResultSet rs = db.executeQuery("SELECT * FROM FAULT WHERE
PROJECTID='"+listenproject+"'");
    try{
        while(rs.next())
        {
            faultn[i]=rs.getString("FAULTID");
            System.out.println(faultn[i]);
            device[i]=rs.getString("DEVICEID");
            object[i]=rs.getString("OBJECTID");
            low[i] = rs.getString("FAULTVALUE");
            up[i] = rs.getString("FAULTVALUE2");
            tocheck[i] =rs.getString("FAULTCHECK");
            i++;
        }
    }catch(SQLException sql)
    { System.out.println("Error connecting Database");

```

จากนั้นทำการเลือกค่าที่ได้ทำการตั้งค่าไว้ในการจัดการความผิดพลาดจากฐานข้อมูลสำหรับโปรเจกต์ที่ผู้เข้ามาใช้บริการได้ทำการเลือกไว้ โดยจัดเก็บค่าเหล่านั้นในรูปแบบของอาร์เรย์ของสายอักขระเพื่อนำมาใช้งานต่อไป โดยเลขชี้ลำดับในอาร์เรย์สายอักขระแต่ละตัวจะเป็นตัวบ่งบอกถึงตัวเมเนจออบเจกต์ ซึ่งแต่ละตัวจะประกอบไปด้วยข้อมูลที่เป็นในการจัดการความผิดปกติ

```

public void run() {
    .....
    while (flag==true) {
    try {
        Thread.currentThread().sleep(Integer.parseInt(refresh));
    } catch (Exception e) { }
    .....
    if(auto==true)
    { Date thisdate = new Date(System.currentTimeMillis());
      Time thistime = new Time(System.currentTimeMillis());
      // ทำการตรวจสอบวันที่ที่ที่ต้องการให้เริ่มทำการจัดการความผิดพลาด เพื่อทำการตั้งค่า flag2
      if(flag2==true||auto==false)
      {
      for(int j=0;j<i;j++)
      {
          mh.constructMessage(device[j],object[j],"GET","", "");
          mh.sendMessage();
          result = mh.receiveMessage();
          if(result.compareTo("*")!=0)
          {
              //ตรวจสอบว่าหากไม่มีการตอบสนองกลับมาจากส่วนของ SNMP Manager
              if(tocheck[j].compareTo("UPPER")==0 &&
              datatype[j].compareTo("integer")==0 )
              {
                  //หากเป็นไปตามเงื่อนไขก็ทำการแจ้งเตือน และ เก็บเหตุการณ์ความผิดพลาดลงฐานข้อมูล
                  else if((tocheck[j].compareTo("LOWER")==0) &&
                  (datatype[j].compareTo("integer")==0))
                  {
                      //หากเป็นไปตามเงื่อนไขก็ทำการแจ้งเตือน และ เก็บเหตุการณ์ความผิดพลาดลงฐานข้อมูล
                      else if((tocheck[j].compareTo("RANGE")==0) &&
                      (datatype[j].compareTo("integer")==0))
                      {
                          //หากเป็นไปตามเงื่อนไขก็ทำการแจ้งเตือน และ เก็บเหตุการณ์ความผิดพลาดลงฐานข้อมูล
                          else if((tocheck[j].compareTo("EQUAL")==0) &&
                          (datatype[j].compareTo("string")==0))
                          {
                              //หากเป็นไปตามเงื่อนไขก็ทำการแจ้งเตือน และ เก็บเหตุการณ์ความผิดพลาดลงฐานข้อมูล
                              else if((tocheck[j].compareTo("EQUAL")==0) &&
                              (datatype[j].compareTo("integer")==0))
                              {
                                  //หากเป็นไปตามเงื่อนไขก็ทำการแจ้งเตือน และ เก็บเหตุการณ์ความผิดพลาดลงฐานข้อมูล

```

เมื่อทำการสั่งให้เธรด (Thread) นี้เริ่มทำงาน มันจะทำการ polling ไปยังอุปกรณ์ต่าง ๆ ตามที่ได้ทำการตั้งค่าไว้เมื่อแรกเริ่ม โดยวิธีการในการโพลลิ่งนั้นจะวนไปตามเมนจออบเจ็กต์

ของอุปกรณ์แต่ละตัวเรียงลำดับตามที่ได้ทำการเลือกขึ้นมาจากฐานข้อมูลโดย ช่วงเวลาของการโพลลิงระหว่างเมเนจอร์ที่ต้องการจัดการความผิดปกตินั้น จะสามารถที่จะกำหนดได้ตามความต้องการของผู้ใช้บริการนอกจากนั้นแล้ว เนื่องจากโปรแกรมระบบจัดการอุปกรณ์นั้นได้ทำการออกแบบตามระบบจัดการเครือข่าย ดังนั้นจึงเพิ่มส่วนประกอบที่ทำหน้าที่ในการเฝ้าฟังแตรป (Trap Listener)

```
public class Traplistener extends JInternalFrame {
    Snmp snmp1 = new Snmp();
    Database db = new Database();
    .....
    private void jbInit() throws Exception {
        .....
        snmp1.addSnmpEventListener(new ipworks.SnmpEventListener() {
            .....
            public void trap(SnmpTrapEvent e) {
                snmp1_trap(e);
            }
        });
        .....
        void snmp1_trap(SnmpTrapEvent e){
            // เมื่อเกิดแตรปเข้ามา ก็จะทำการเอา Trap event นั้น มาประมวลผล และจัดเก็บไปสู่ตารางเหตุการณ์ผิดปกติ
            .....
            try {
                snmp1.setActive(!snmp1.isActive());
            } catch (IPWorksException ipwe) {
                JOptionPane.showMessageDialog(this, ipwe.getMessage());
            }
            // เพื่อปิดและเปิดการเฝ้าฟังแตรป โดยใช้ Event ของการกดปุ่ม
        }
    }
}
```

จากทฤษฎีที่ได้กล่าวไว้ว่าวิธีการที่ใช้ในการจัดการเครือข่ายประกอบไปด้วย 2 วิธีการคือการโพลลิงไปยังอุปกรณ์เป็นระยะ ๆ และ วิธีที่สองคือการใช้แตรป ดังนั้นเพื่อควมมีประสิทธิภาพในการทำงานนั้น โปรแกรมระบบจัดการอุปกรณ์จึงได้พัฒนาให้โปรแกรมมีความสามารถในทั้ง 2 วิธีการ

4.8 การพัฒนาส่วนการจัดเก็บข้อมูลลงฐานข้อมูล (Data logging)

จากในบทที่ผ่านมาได้กล่าวถึงการออกแบบกลไกที่นำมาจัดการว่า เราจะจัดเก็บข้อมูลที่ต้องได้อย่างไรให้สามารถจัดเก็บได้อย่างเป็นหมวดหมู่ และเป็นระบบ ระเบียบ เพื่อที่จะสามารถเรียกขึ้นมาใช้งานได้ในภายหลัง

```

ResultSet rs2 = db.executeSQLQuery("SELECT * FROM
SUPPORTGROUP WHERE PROJECTID='"+projectname+"'");
int x=0;
try{
while(rs2.next())
{
name[x]=rs2.getString(2);
x++;
}
}catch(SQLException sql) {}

```

จัดเก็บข้อมูลขึ้นมา จากนั้นนำรายชื่อกลุ่มเหล่านั้นส่งต่อไปยังคลาสซึ่งเป็นเธรด (Thread) ซึ่งทำหน้าที่ในการเก็บข้อมูลที่ได้อมาจากการตอบสนองของอุปกรณ์ไปเก็บอย่างถูกต้อง


```

public class Logging extends Thread{
.....
public Logging(String pname,String[] group,String rrate) {
// constructor
.....
public void run()
{
.....
ResultSet rs4 = db.executeQuery("SELECT * FROM MONITORGROUP
WHERE GROUPID='"+loggroup[i]+"");
try{
    while(rs4.next())
    {
        String devicename = rs4.getString(2);
        String objectname = rs4.getString(3);
        statement= statement.concat(devicename+objectname+",");
        mh.constructMessage(devicename,objectname,"GET","", "");
        mh.sendMessage();
        temp=mh.receiveMessage();
        value[w]= temp;

        if(value[w].compareTo("")!=0)
        {
            w++;
        }
    }
} catch(SQLException sql)
{}
String statement2="";
for(int m=0;m<gnum;m++)
{
    statement2= statement2.concat(value[m]+"", "");
}
if(temp.compareTo("")!=0)
{
    db.executeUpdate("INSERT INTO
"+projectname+"_"+loggroup[i]+"_"+LOG+"
(PROJECTID,GROUPID,"+statement+"THEDATE,THETIME)VALUES
('"+projectname+"','"+loggroup[i]+"','"+statement2+(new
Date(System.currentTimeMillis()).toString())+"',(new
Time(System.currentTimeMillis()).toString())+'");
}
}

```

ข้อมูลของอุปกรณ์จะถูกจัดเก็บไปยังตารางที่ได้มีการสร้างไว้ ในขณะที่ได้มีการตั้งค่าในตอนเริ่มแรกของการใช้งาน